

Scaling & Optimizing BLAST (Human Chromosome 21)

Last class we saw how to manually BLASTed genes on the NCBI website and then you start working on your own documented `run_blast.sh` script to run it in the server. Today we will test if your script works and if not, we will start learning how to debug or fix your own scripts. Once you are 100% sure that it works in one gene, we will go to **scale** it (run it on many genes with a loop) and then **optimize** it (download our data with a better tool rather than downloading with `curl`). You will learn about NCBI `datasets`.

:rotating_light: **Remember** to document each step in a separate script document, following the best practices we covered in class.

[!NOTE] Today we will need the following documents:

- Have your `bash_loops_conditionals_chr21.md` document and your chromosome 21 script ready for review.

Today we will try to work at our own pace, I will explain step by step, but help me to support your classmates. Helping debugging a peer's code syntax, is the best way to solidify your own knowledge.

Activity goals:

- Optimize how we use our **variables** to make a script reproducible.
- Practice `while read` loop to run BLAST on a list of genes.
- Use a conditional to check your inputs before running an analysis.
- Practice script documentation.

In the Chromosome 21 exercise you created `10_genes.txt`, a list of 10 protein-coding genes from chromosome 21 with their RefSeq accession numbers. Today, those 10 genes are our input data.

For today's class exercise, we will:

1. Add variables to your `run_blast.sh` script.
2. Test your script with ONE gene only.
3. BLAST all 10 genes from the Chromosome 21 exercise against chromosome 21 database, in a single loop.
4. Decide if we still need the loop.
5. Create and document your scripts.

[!TIP] Syntax Remember that everytime you see this "Syntax section" you will find the basic syntax for the commands that we are using today so it is easier for you to revisit them as needed.

Table of Contents

1. Prepare your working environment	4
Test your bash script:	4
:handshake: Please if you were able to run your script, pause a little bit before continuing and check with your classmates around you to see if someone is struggling and offer your help if needed. Helping to debug scripts is one of the best ways of learning.	5
2a. Reference database	6
3. BLAST 10 genes with a loop	7
3a. Add a conditional	8
To get credit:	9

1. Prepare your working environment

:warning: Remember that you are working in Tule, not at your computer anymore!

To access Tule:

```
ssh -p XXXX username@tule.usfca.edu
```

#XXXX = Is the port number I gave you

#username = substitute "username" for the information that I gave you

To check that you are inside of Tule, your terminal should say something like this:

```
username@bio-toolkit:~$
```

If you haven't do this, create your main directory `btec_640`.

Test your bash script:

Your script should already have the instructions to create your directories.

:point_right: Before we start, look at your script and make sure that you working directory will look like this:

:open_file_folder: `btec_640`

- `:file_folder: input_data`
- `:file_folder: analysis`
- `:file_folder: output_dir`

:point_right: Connect your VScode to Tule server (check `Instructions_for_Tule.md` file if needed)

:point_right: On VScode, move to the `btec_640` directory create a new file (make sure you are inside Tule), and

paste your blast script from last class.

:point_right: Let's going to use the file: `LSS.fasta` as our input, assign it to your input variable, and save it with the `.sh` extension (for example `run_blast.sh`).

:point_right: Run it: `sh run_blast.sh`

Did it run succesfully? Yes/No what happened?

If it failed, what was the issue and what do you need to fix it?

Type your answer here:

:point_right: Let's get the data, you already have your script for this. On your VScode, copy and paste your `Chromosome21.sh` script from your computer to your Tule environment.

:warning: Your will need to modify/add the variables of your Working directories, otherwise is not going to work.

:point_right: You will need to: Generate the `10_genes.txt` file again, and download your 10 fasta files (But this time save these at `input_data` directory, modify your script as needed)

:handshake: Please if you were able to run your script, pause a little bit before continuing and check with your classmates around you to see if someone is struggling and offer your help if needed. Helping to debug scripts is one of the best ways of learning.

:point_right: Run again: `sh run_blast.sh` with `LSS.fasta` as your input variable again. What happened now?

Type your answer:

It spit out nothing because it has no database

2a. Reference database

We need a reference for blast, or use the entire ncbi dataset by adding the flag `-remote`, but this will take a long time for this class. So let's provide blast our own database. Let's download the chromosome 21 fasta file from human as our database:

:warning: before run it, look carefully at the flag `-o`, make sure you run this command in the right path so it can immediately find the directory `input_data`

:point_right: Download the data

```
curl -o input_data/chr21.fa.gz "https://hgdownload.soe.ucsc.edu/goldenPath/hg38/chromosomes/chr21.fa.gz"
cd input_data
gunzip chr21.fa.gz
```

:point_right: Create the database for blast. Check at the NCBI [documentation](#), you will see the syntax for the command `makeblastdb`.

WE NEED TO RUN `makeblastdb` before `blastn`, edit your `bash_script` adding this line right before `blastn`.

[!TIP] Syntax: `makeblastdb` (Create a custom database from a multi-FASTA file)

```
makeblastdb -in mydb.fsa -dbtype XXX -out output_database
```

- *in your input data in this case chr21.fa*
- *dbtype where XXX is the type of data that you are working with: nucl or prot (nucleotide or protein respectively)*
- *out the filename for the the database is going to be created*

:question: What option will you use for the `-dbtype` flag? `nucl` or `prot`? And how did you know?

Type your answer here

`nucl` because were doing `blastn`

:question: Type here the command that you would add to your script, and the variable that you will create to make it scalable.

:warning: Make sure you are using your working directories variables so your script can find `chr21.fa` and `makeblastdb` output is saved inside your `output_data` directory.

Type your command here:

```
makeblastdb -in $WORKDIR/$INPUTDIR/chr21.fa -dbtype nucl -out $WORKDIR/$ANALYSISDIR/output_database
```

:point_right: Add your new command to your script before `blastn` and run all your script again on `LSS.fasta`

:question: Did it work?

:question: Go to your `output_data` directory and type `ls`. Which files were generated? Explore the files and try to understand the content.

Paste here the filenames followed by a brief explanation of what did you observe

`blast_result.txt`

Now our script works for one gene, but imagine if we need to analyze 100 genes, or 20,000 genes. Let's try to optimize this with a `loop`

3. BLAST 10 genes with a loop

You already know the `while read` loop from the Chromosome 21 exercise. As a reminder:

[!TIP] Syntax: `while read` (loop over a file, line by line)

```
while read -r var1 var2
do
  COMMANDS_USING_${var1}_AND_${var2}
done < INPUT_FILE
```

:question: Check your `bash_loops_conditionals_chr21.md` document and the `while read` syntax above.

:question: How would you structure your `while read` syntax for your `run_bash.sh` script, your input file for this loop, would be the `10_genes.txt` file that we generated with our chr21 exercise, that should look like this:

```
LSS NM_001145436.2
SPATC1L NM_001142854.2
FTCD NM_001320412.2
GART NM_000819.5
DNAJC28 NM_017833.5
TMEM50B NM_006134.7
IFNGR2 NM_005534.4
C21orf62 NM_001162495.3
PAXBP1 NM_013329.4
BTG3 NM_006806.5
```

```
#Type your while read command here USING YOUR CURRENT VARIABLES
```

Check the results:

```
ls -l analysis/*.tsv
wc -l analysis/*.tsv
```

3a. Add a conditional

Analysis or downloads can fail. There is no point in running BLAST on an empty file, so let's check first.

[!TIP] Syntax: `if / else with -s`

```
if [ -s "$file" ]
then
  COMMANDS_IF_FILE_EXISTS_AND_IS_NOT_EMPTY
else
  COMMANDS_IF_NOT
fi
```

Add this to your loop, right after the `curl` line and before `blastn`:

```
if [ ! -s "input_data/${gene}.fasta" ]
then
  echo "Error :( ${gene}.fasta is empty. Skipping..."
  echo "$gene $accession" >> failed_downloads.txt
  continue
fi
```

- `!` means **NOT**, so `[ ! -s file ]` reads as "if the file is empty or does not exist".
- `continue` skips the rest of the loop for this gene and moves on to the next one.

To get credit:

Only upload this document on `pdf` to canvas, under the assignment `blast_tule`